



Data Structures

(Course Code: 25B11CI318)

Binary Tree

Mr. Sandeep Kumar Patel

Assistant Professor

Jaypee University of Information Technology

Waknaghat, Solan, HP-173234

What is a Binary Tree?

Definition

A **binary tree** is defined recursively on a finite collection of nodes. It is either empty, or consists of a root together with a **left binary subtree** and a **right binary subtree**. The root and the two subtrees are mutually disjoint.

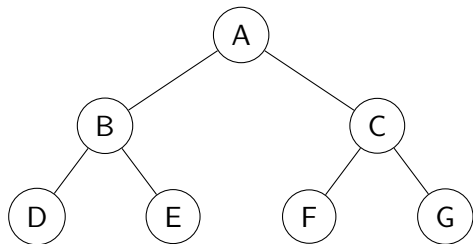
For every node v ,

$$\text{Number of children of } v \in \{0, 1, 2\}$$

- Either subtree may be empty.
- Left and right positions are distinct.

Definition adapted from Cormen et al., *Introduction to Algorithms*, 3rd ed., Appendix B.5.

Basic Example of a Binary Tree



- Root: *A*
- Children of *A*: *B, C*
- Children of *B*: *D, E*
- Children of *C*: *F, G*
- Leaves: *D, E, F, G*
- Internal nodes: *A, B, C*

Basic Terminology

Root: Distinguished node with no parent.

Parent: Immediate predecessor of a child.

Child: A node directly connected below its parent.

Siblings: Nodes that share the same parent.

Leaf: A node with no children.

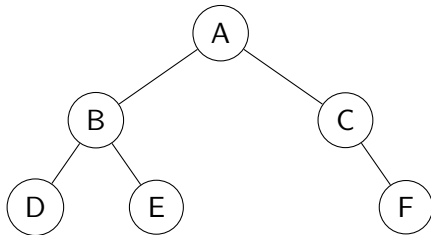
Internal node: A node with at least one child.

Degree: Number of children, in rooted-tree terminology.

Subtree: A node and all its descendants, with their connecting edges.

Level, Depth and Height

We use **level = depth**, with the root at level 0.



- A: depth 0
- B, C: depth 1
- D, E, F: depth 2
- Height of B: 1
- Height of the tree: 2

Note

Height counts edges here. A leaf has height 0. A path containing h edges contains $h + 1$ nodes.

Height and Depth in a Binary Tree

Definition of Depth

The **depth of a node** is the number of edges on the unique path from the root to that node.

$$d(\text{root}) = 0, \quad d(\text{child}) = d(\text{parent}) + 1$$

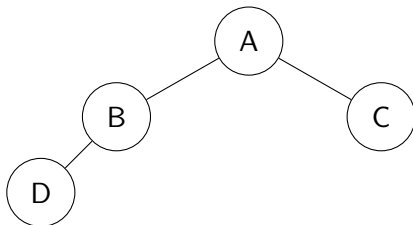
Definition of Height

The **height of a node** is the number of edges on the longest downward path from that node to a leaf.

$$h(v) = 1 + \max\{h(v_{\text{left}}), h(v_{\text{right}})\}$$

For this recurrence, assign an empty subtree height -1 . Thus a leaf has height 0.

Example: Height and Depth



Node	Depth	Height
<i>A</i>	0	2
<i>B</i>	1	1
<i>C</i>	1	0
<i>D</i>	2	0

Understanding Height and Depth

In the preceding tree, the longest root-to-leaf path is

$$A \rightarrow B \rightarrow D.$$

It contains **two edges**, so

$$h(A) = 2.$$

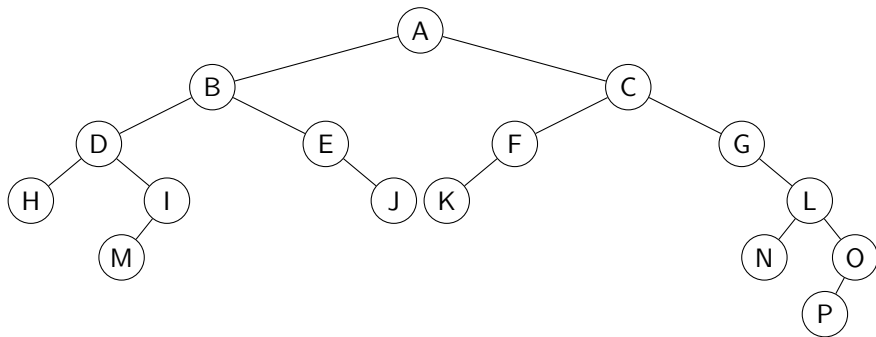
Important Difference

Depth: distance from the root to the node

Height: longest distance from the node to a leaf

Node *C* has depth 1 but height 0.

Question: Height and Depth



Root depth is 0; height and depth count edges.

Questions

For the binary tree shown above, determine:

- 1 The **depths** of I, M, L, O, P .
- 2 The **heights** of B, C, D, G, L .
- 3 The **height of the entire tree**, identifying a longest root-to-leaf path.
- 4 The node with **maximum depth**.
- 5 All **leaf nodes**.

Solution: Height and Depth

Depth

$$d(I) = 3, \quad d(M) = 4,$$

$$d(L) = 3, \quad d(O) = 4,$$

$$d(P) = 5.$$

Solution: Height and Depth

Depth

$$\begin{aligned}d(I) &= 3, & d(M) &= 4, \\d(L) &= 3, & d(O) &= 4, \\d(P) &= 5.\end{aligned}$$

Height

$$\begin{aligned}h(B) &= 3, & h(C) &= 4, \\h(D) &= 2, & h(G) &= 3, \\h(L) &= 2.\end{aligned}$$

Tree Height

A longest root-to-leaf path is

$$A \rightarrow C \rightarrow G \rightarrow L \rightarrow O \rightarrow P.$$

It contains five edges:

$$h(A) = 5.$$

Other Answers

Maximum-depth node: P .

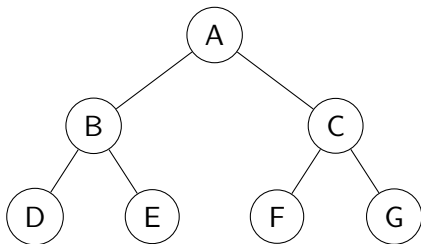
Leaf nodes:

$$H, M, J, K, N, P.$$

Types of Binary Trees

- **Full:** every node has either 0 or 2 children.
- **Complete:** all levels before the last are full; the last level occupies the leftmost positions.
- **Perfect:** every internal node has two children and all leaves have the same depth.
- **Height-balanced (AVL condition):** left and right subtree heights differ by at most one at every node.
- **Skewed:** nodes form a single path using only left-child links or only right-child links.

Full Binary Tree



Property

Every internal node has exactly two children.
Leaves need not have equal depths.

With I internal nodes and L leaves:

$$N = I + L, \quad N - 1 = 2I.$$

Therefore,

$$L = I + 1, \quad N = 2I + 1.$$

The illustrated tree is also perfect.

Perfect Binary Tree

All levels are completely filled. For height h :

$$N = 2^{h+1} - 1$$

$$L = 2^h$$

$$I = N - L = 2^h - 1$$

Example

For $h = 3$:

$$N = 15, \quad L = 8, \quad I = 7.$$

A perfect binary tree is both full and complete.

Important Properties of Binary Trees

Assume root level 0 and height measured in edges.

- Maximum number of nodes at level i :

$$2^i$$

- For a non-empty tree of height h :

$$h + 1 \leq N \leq 2^{h+1} - 1$$

- A non-empty tree with N nodes has:

$$E = N - 1 \text{ edges}$$

What is Tree Traversal?

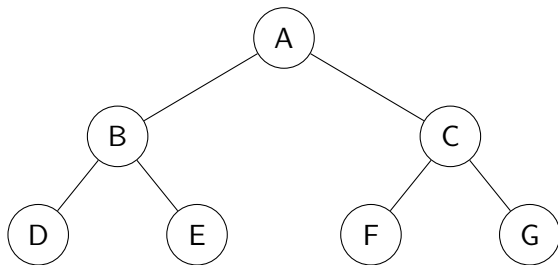
Definition

A traversal visits every node exactly once in a specified order. A **visit** performs an operation such as printing or processing the node's value.

Main traversal methods:

- ① Preorder
 - ② Inorder
 - ③ Postorder
 - ④ Level order
- First three: **depth-first traversals (DFS)**.
 - Level order: **breadth-first traversal (BFS)**.

Example Tree Used for Traversals



The tree stays unchanged; only the order of visits changes.

Preorder Traversal

Rule

Root $\rightarrow$ Left subtree $\rightarrow$ Right subtree

NLR

Here, N denotes the current node.

Apply the rule recursively at every node:

$A \rightarrow B \rightarrow D \rightarrow E \rightarrow C \rightarrow F \rightarrow G.$

Therefore,

Preorder: $A, B, D, E, C, F, G.$

Preorder Algorithm

```
PREORDER(root)
{
    if (root == NULL)
        return;

    VISIT(root);
    PREORDER(root.left);
    PREORDER(root.right);
}
```

Idea

Visit each node before its descendants. The empty subtree is the recursion's base case.

Preorder: Step-by-Step Trace

For the tree rooted at A :

- ① Visit root A .
- ② Enter the left subtree rooted at B .
- ③ Visit B , then its left leaf D , then right leaf E .
- ④ Return from the completed left subtree.
- ⑤ Enter the right subtree rooted at C .
- ⑥ Visit C , then its left leaf F , then right leaf G .

Final order:

A, B, D, E, C, F, G

.

Inorder Traversal

Rule

Left subtree $\rightarrow$ Root $\rightarrow$ Right subtree

LNR

Visit the root after its left subtree and before its right.

For the example tree:

$D \rightarrow B \rightarrow E \rightarrow A \rightarrow F \rightarrow C \rightarrow G.$

Therefore,

Inorder: $D, B, E, A, F, C, G.$

Inorder Algorithm

```
INORDER(root)
{
    if (root == NULL)
        return;

    INORDER(root.left);
    VISIT(root);
    INORDER(root.right);
}
```

Important

Inorder traversal of a **BST** returns keys in sorted order. An arbitrary binary tree has no such guarantee.

Postorder Traversal

Rule

Left subtree $\rightarrow$ Right subtree $\rightarrow$ Root

LRN

Visit the root only after completing both subtrees.

For the example tree:

$D \rightarrow E \rightarrow B \rightarrow F \rightarrow G \rightarrow C \rightarrow A.$

Therefore,

Postorder: $D, E, B, F, G, C, A.$

Postorder Algorithm

```
POSTORDER(root)
{
    if (root == NULL)
        return;

    POSTORDER(root.left);
    POSTORDER(root.right);
    VISIT(root);
}
```

Idea

Process descendants before their parent. This order supports subtree calculations and tree deletion.

Level Order Traversal

Rule

Visit nodes in increasing order of depth, from left to right within each level.

For the example tree:

Level 0: *A*

Level 1: *B, C*

Level 2: *D, E, F, G*

Hence,

Level order: *A, B, C, D, E, F, G*.

A **FIFO queue** holds nodes awaiting their visit.

Level Order Algorithm

```
LEVELORDER(root)
    if root == NULL
        return

    create empty queue Q
    ENQUEUE(Q, root)

    while Q is not empty
        node = DEQUEUE(Q)
        VISIT(node)

        if node.left != NULL
            ENQUEUE(Q, node.left)
        if node.right != NULL
            ENQUEUE(Q, node.right)
```

Comparison of Traversals

Traversal	Rule	Output
Preorder	Root, Left, Right	<i>A, B, D, E, C, F, G</i>
Inorder	Left, Root, Right	<i>D, B, E, A, F, C, G</i>
Postorder	Left, Right, Root	<i>D, E, B, F, G, C, A</i>
Level order	Increasing depth	<i>A, B, C, D, E, F, G</i>

For DFS, “left” and “right” mean traversing the entire corresponding subtree recursively.

Quick Memory Trick

Preorder: Root before Subtrees

Inorder: Root between Subtrees

Postorder: Root after Subtrees

Traversal	Short form	Root visit
Preorder	NLR	Before both subtrees
Inorder	LNR	Between the subtrees
Postorder	LRN	After both subtrees

Time and Space Complexity

For N nodes and constant-time work per visit:

$$T(N) = \Theta(N).$$

Recursive DFS uses a stack proportional to path length:

$$\text{Auxiliary space} = O(h + 1).$$

- Balanced tree: $O(\log N)$ stack space.
- Skewed tree: $O(N)$ stack space.
- Level order: $O(w)$ queue space, where w is maximum level width; worst case $O(N)$.

These bounds exclude tree storage and separately stored output.

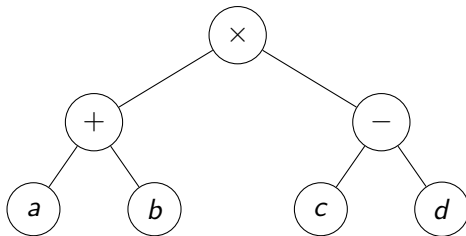
Applications of Traversals

Traversal	Typical applications
Preorder	Prefix expressions; copying trees; serialization with null markers or equivalent structural information
Inorder	Sorted BST output; infix expressions with appropriate parentheses
Postorder	Freeing a tree; expression evaluation; calculations that depend on subtree results
Level order	Breadth-first search; processing by depth; checking completeness

Expression Tree Example

Consider the expression

$$(a + b) \times (c - d).$$



Internal nodes represent binary operators; leaves represent operands.

Expression Tree Traversals

For the expression $(a + b) \times (c - d)$:

- **Preorder / Prefix:**

$\times \quad + \quad a \quad b \quad - \quad c \quad d$

- **Inorder with parentheses / Infix:**

$(a + b) \times (c - d)$

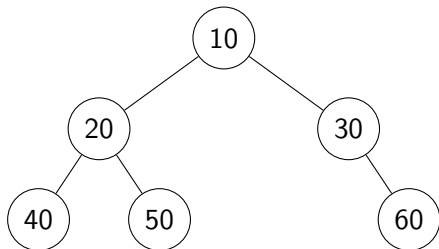
- **Postorder / Postfix:**

$a \quad b \quad + \quad c \quad d \quad - \quad \times$

Plain inorder output does not include parentheses. They must be inserted to preserve the tree's grouping.

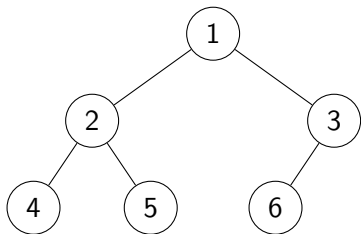
Solved Example

Find all traversals of the following tree.



- **Preorder:**
10, 20, 40, 50, 30, 60
- **Inorder:**
40, 20, 50, 10, 30, 60
- **Postorder:**
40, 50, 20, 60, 30, 10
- **Level order:**
10, 20, 30, 40, 50, 60

Another Practice Example



Determine each traversal before checking:

- ① Preorder
- ② Inorder
- ③ Postorder
- ④ Level order

Answers:

Preorder: 1, 2, 4, 5, 3, 6

Inorder: 4, 2, 5, 1, 6, 3

Postorder: 4, 5, 2, 6, 3, 1

Level order: 1, 2, 3, 4, 5, 6

Thank You